StyleGAN

——深度学习第1组汇报

汇报人　：xxxxx

代码精读：xxxxx、xxxxx

论文精读：xxxxx

结果复现：xxxxx、xxxxx

PPT制作：xxxxx

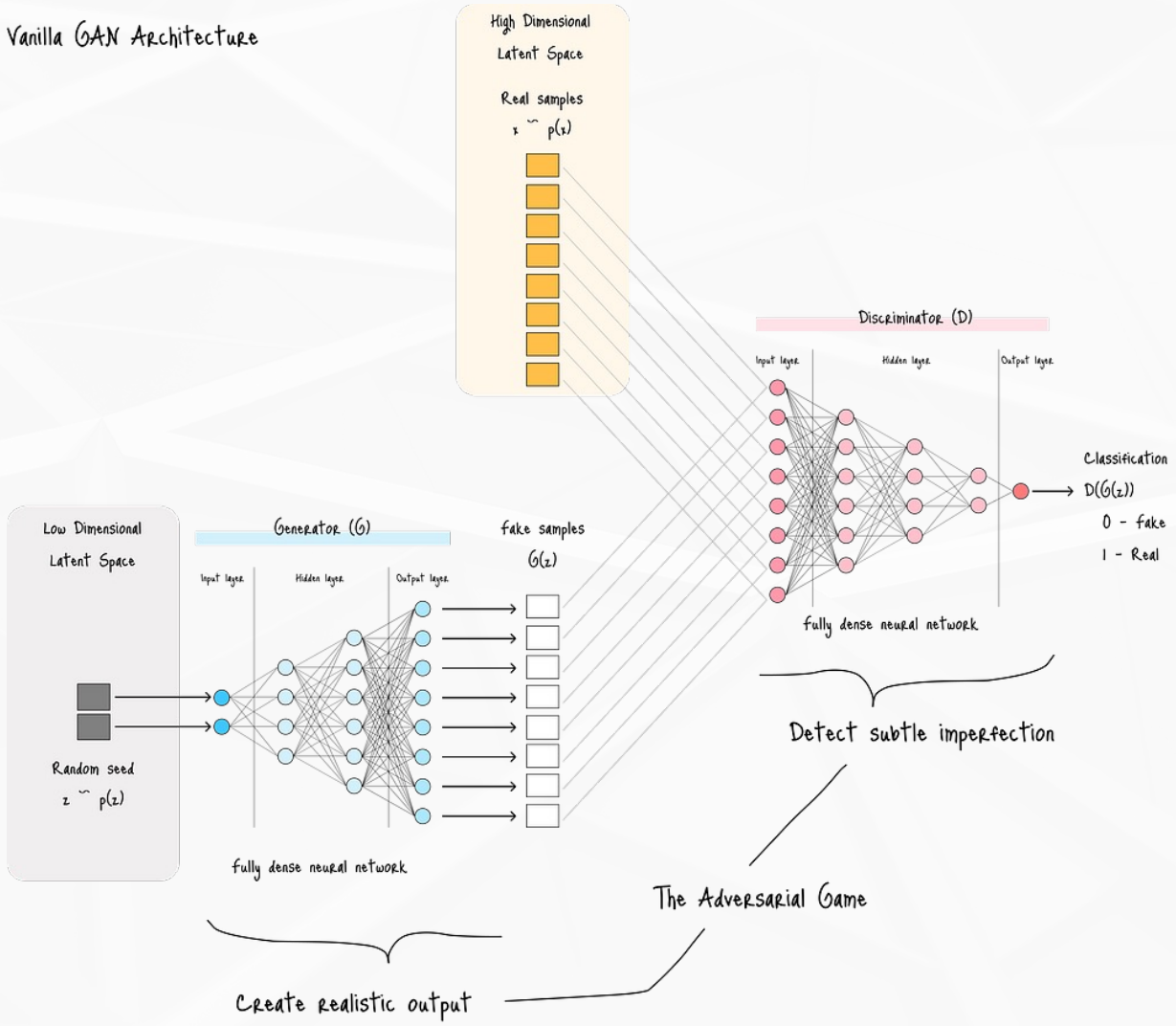
西北工業大學

NORTHWESTERN POLYTECHNICAL UNIVERSITY

CONTENT

目录

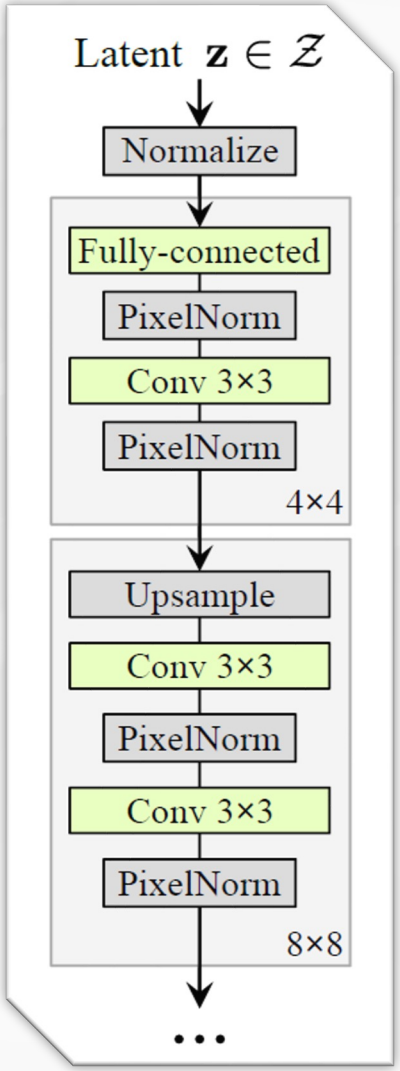
01 What's StyleGAN —— GAN



通过生成器和判别器的博弈，让模型学会生成"以假乱真"的数据

01 What's StyleGAN —— Motivation

西北工業大學



- **潜变量 z 与图像特征高度纠缠**

随机向量 z 的某个维度往往影响多种属性

⇒ **模型缺乏可控性**

- **图像的生成过程是不可见的黑盒**

- 内部特征映射与输出之间的关系不透明
- 难以理解各层卷积特征对应的视觉含义
- 无法区分高层语义与低层纹理的作用

⇒ **模型缺乏可解释性**

- **潜空间 $\mathcal{Z}$ 插值质量差**

- $\mathcal{Z}$ 与图像语义之间的映射高度非线性
- 插值过程可能穿越数据稀疏区导致突变

⇒ **潜空间非线性、不平滑**

Traditional --> **把随机向量 z 直接输入网络的第一层，然后通过卷积层逐步上采样生成图像。**

01 What's StyleGAN —— Histroy of StyleGAN

西北工業大學

2019 StyleGAN1

- 通过8个全连接层实现特征间的**解耦**
- 每一层的仿射变换都会为该层注入**独立**的风格控制信号
- AdaIN**指导**风格生成，注入**随机**噪声来丰富细节
- 逐步**上采样**，直到生成高分辨率图像（1024*1024）

2020 StyleGAN2

- 弃用AdaIN，直接在卷积**权重**上施加风格**调制**。

避免特征统计失真与层间风格不连续问题

- 由“先生成图像再插值”改为“直接在特征图上上采样”

显著减少伪影
提高生成的平滑性与一致性

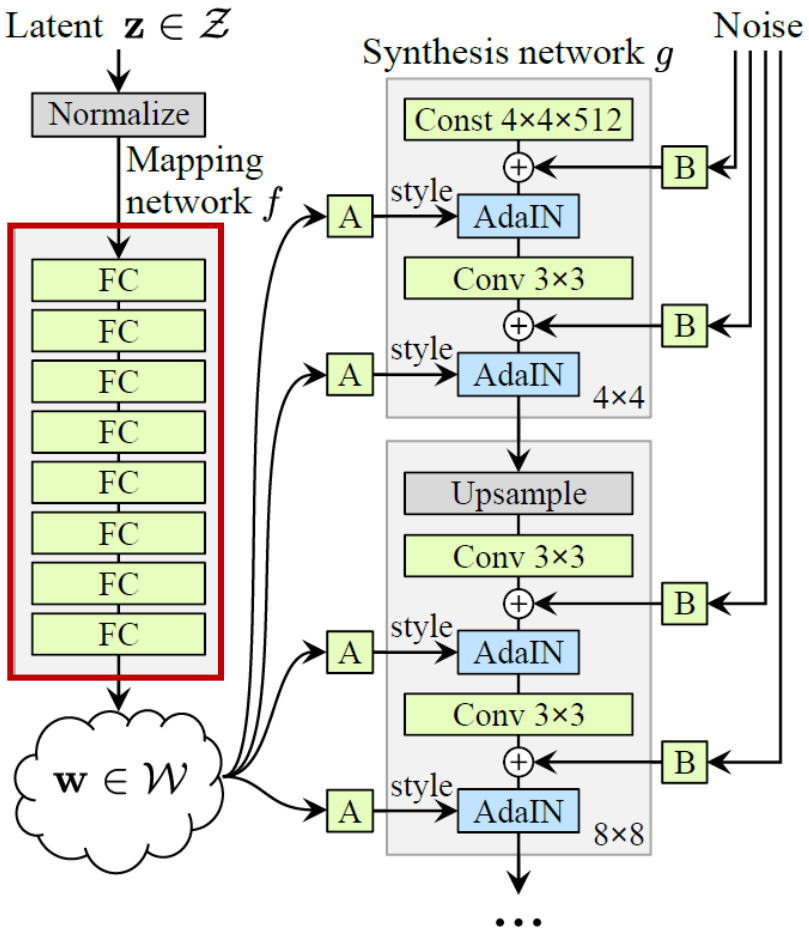
2021 StyleGAN3

- 从信号的角度出发，修改上采样方案
- 增加合理的低通滤波方案

生成图像更平滑、自然，在旋转等变换时细节更稳定
改善空间连续性，图像在轻微几何变化下保持一致性

西北工業大學

Style-based generator



Our code

```python
class MappingNetwork(nn.Module):
    def __init__(self, z_dim, w_dim):
        super().__init__()
        self.mapping = nn.Sequential(
            PixelNorm(),
            WSLinear(z_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
            nn.ReLU(),
            WSLinear(w_dim, w_dim),
        )
    def forward(self, x):
        return self.mapping(x)
```

```python
class WSLinear(nn.Module):
    def __init__(
        self, in_features, out_features,
    ):
        super(WSLinear, self).__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.scale = (2 / in_features)**0.5
        self.bias = self.linear.bias
        self.linear.bias = None
        # 初始化线性层
        nn.init.normal_(self.linear.weight)
        nn.init.zeros_(self.bias)
    def forward(self, x):
        return self.linear(x * self.scale) + self.bias
```

- **WSLinear：“带权重标准化”的线性层**
  - self.scale：保证输入经 ReLU 后方差恒定的关键系数，防止爆炸或衰减
- **MappingNetwork：$\mathcal{Z}$ -> $\mathcal{W}$**
  - 把 $\mathcal{Z}$ 映射到一个更平滑且解耦的潜空间 $\mathcal{W}$，消除 $\mathcal{Z}$ 的原始分布带来的非线性扭曲

西北工业大学

PPL 感知路径长度

$$l = \sum d(G(\text{interp}(t_i)), G(\text{interp}(t_i + \epsilon)))$$

- Interp(·)：返回两个潜向量 z_i, z_{i+1} 间的插值点
- G(·)：返回由生成器构建的图像
- d(·)：感知距离函数，用 VGG 特征差来近似人类感知差异

Which means: 在潜空间上走

⇩

在图像空间上测量感知距离

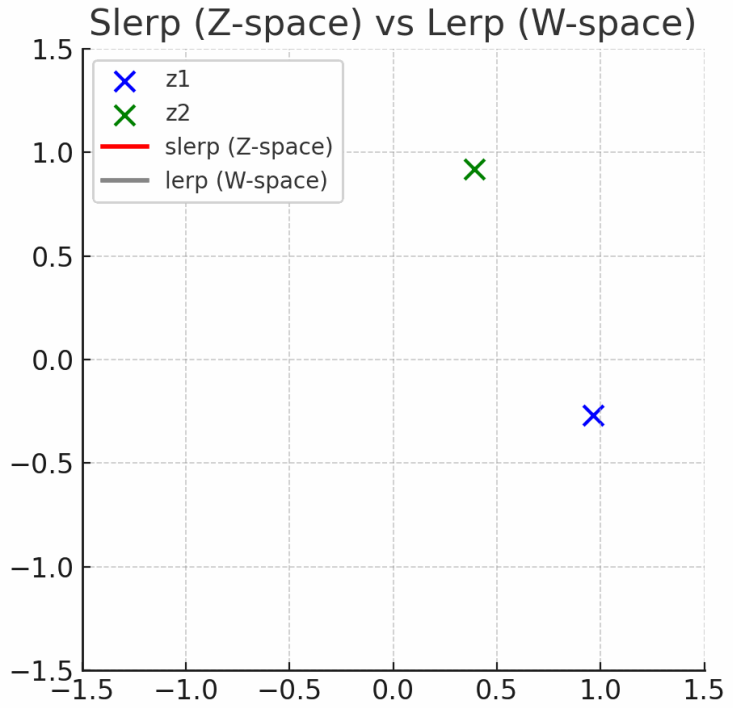
⇩

用 VGG 感知特征衡量

⇩

潜空间越线性 ⇒ 图像感知变化越平滑 ⇒ PPL 越小

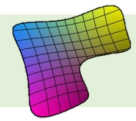
PPL Calculate & Display



$\mathcal{Z}$ – 球面线性插值

$$l_Z = E\left[\frac{1}{\varepsilon^2} d(G(z(t)), G(z(t+\varepsilon)))\right]$$

$\mathcal{W}$ – 线性插值



$$l_W = E\left[\frac{1}{\varepsilon^2} d(G(w(t)), G(w(t+\varepsilon)))\right]$$

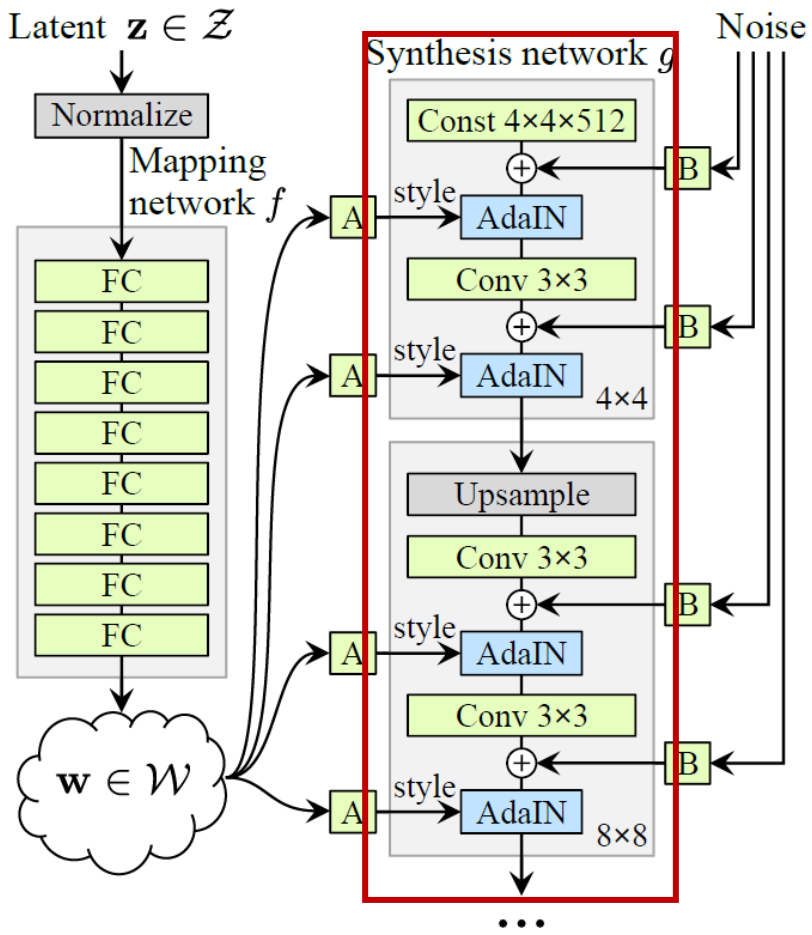
由左图不难看出，潜空间 $\mathcal{W}$ 下的 PPL 更小，意味着它更平滑、更解耦、更具有线性

Mapping 解决了传统模型缺乏可控性的问题

02 How&Why StyleGAN1 Works —— Progressive

西北工業大學

Style-based generator



Our code

```python
# 从对应于我们在配置中设置的图像大小的步骤开始
step = int(log2(START_TRAIN_AT_IMG_SIZE / 4))
for num_epochs in PROGRESSIVE_EPOCHS[step:]:
    alpha = 1e-5  # 从非常低的alpha值开始
    loader, dataset = get_loader(4 * 2 ** step)
    print(f"Current image size: {4 * 2 ** step}")
    for epoch in range(num_epochs):
        print(f"Epoch [{epoch+1}/{num_epochs}]")

        # 新层与旧层的融合比例
        alpha = train_fn(
            critic,
            gen,
            loader,
            dataset,
            step,
            alpha,
            opt_critic,
            opt_gen
        )

        if epoch % 1 == 0:  # 每1个epoch保存一次
            save_models(gen, critic, step, epoch)
    generate_examples(gen, step)
    step += 1  # 进展到下一个图像大小
```

继承自 ProGAN 的渐进式训练

高分辨率下直接训练 ⟶ 生成器和判别器容易失衡

由低到高渐进训练 ⟶

早期参数量少，计算量小，模型快速捕捉整体特征

随着分辨率提升，网络只需学习更多细节特征

相邻分辨率层之间的平滑融合

upscaled
旧层生成的低分辨率图像

＋

generated
新层生成的高分辨率图像

```python
def fade_in(self, alpha, upscaled, generated):
    # alpha应该是[0, 1]范围内的标量，并且 upscale.shape == generated.shape
    return torch.tanh(alpha * generated + (1 - alpha) * upscaled)
```

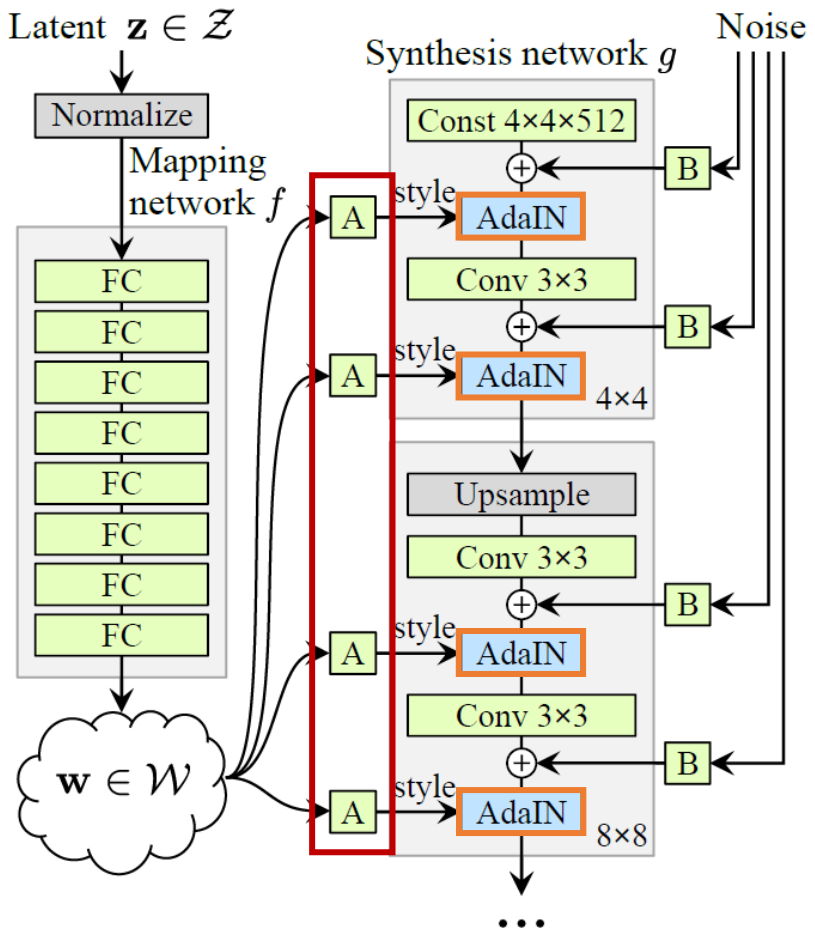
```python
class Generator(nn.Module):
    def forward(self, noise, alpha, steps):
        x = self.initial_adain1(self.initial_noise1(self.starting_constant), w)
        x = self.initial_conv(x)
        out = self.initial_adain2(self.leaky(self.initial_noise2(x)), w)
        if steps == 0:
            return self.initial_rgb(x)
        for step in range(steps):
            upscaled = F.interpolate(out, scale_factor=2, mode="bilinear")
            out = self.prog_blocks[step](upscaled, w)
```

生成器中的上采样 双线性插值

平滑自然，减少锯齿

西北工業大學

Style-based generator



Our code

```python
class AdaIN(nn.Module):
    def __init__(self, channels, w_dim):
        super().__init__()
        self.instance_norm = nn.InstanceNorm2d(channels)
        self.style_scale = WSLinear(w_dim, channels)
        self.style_bias = WSLinear(w_dim, channels)
    def forward(self, x, w):
        x = self.instance_norm(x)
        style_scale = self.style_scale(w).unsqueeze(2).unsqueeze(3)
        style_bias = self.style_bias(w).unsqueeze(2).unsqueeze(3)
        return style_scale * x + style_bias
```

Affine Transform 与 AdaIN 在逻辑设计中是包含关系

⇩

每一层的 Affine Transform 独立负责该层特征的风格控制
每一层的 AdaIN 在指导风格生成时也是独立的

西北工业大学

Our outcome of every step

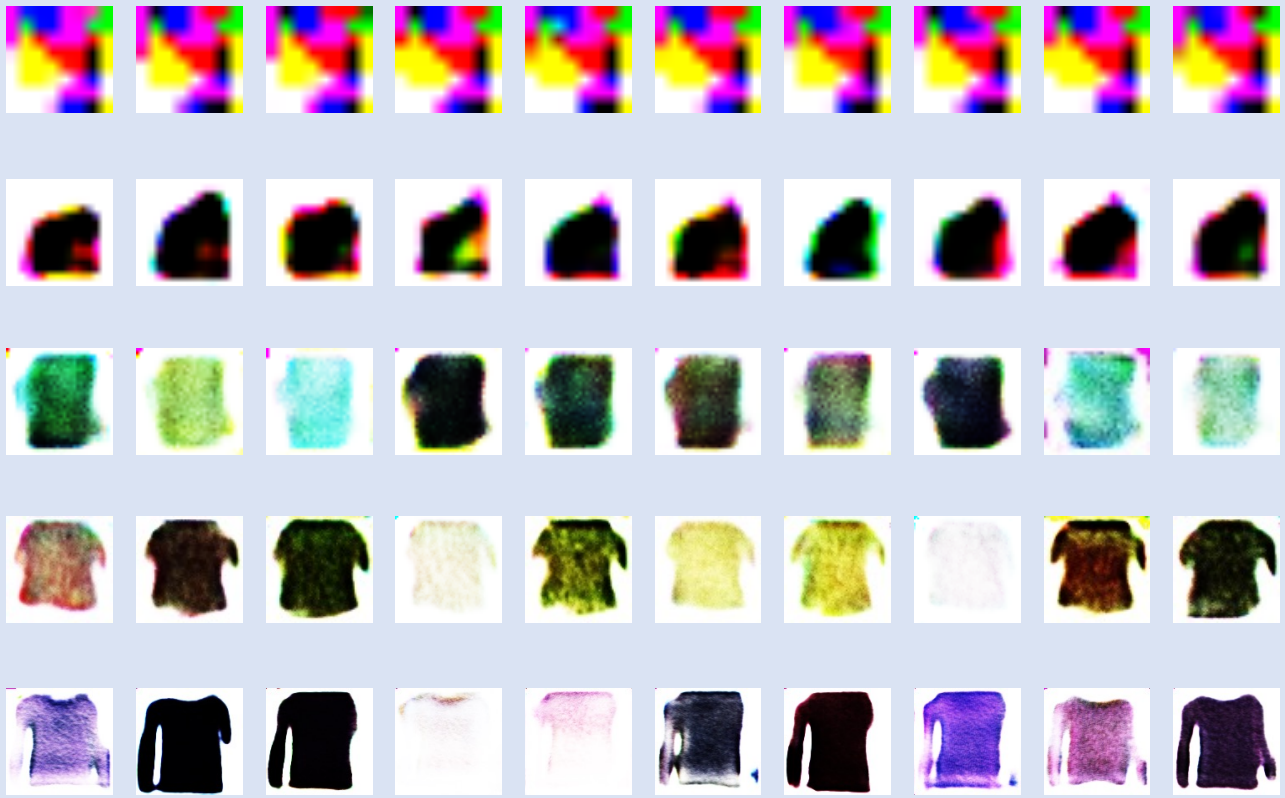
Step1: (4*4)

Step2: (16*16)

Step3: (32*32)

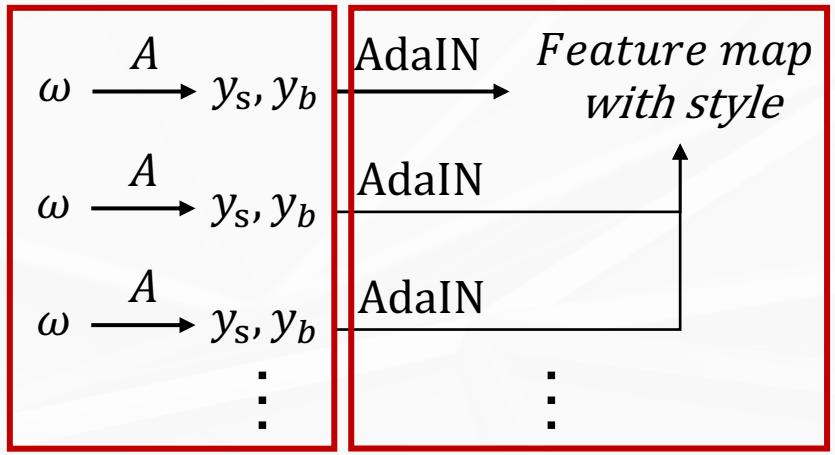
Step4: (64*64)

Step5: (128*128)



PS：为了方便展示，将图片放缩至同一大小，实际大小见其像素尺寸

How Affine Transform works?



分层风格控制　　风格混合

粗粒度/低频特征：

- 形状
- 姿势
- 头部轮廓
- 总体光照方向等

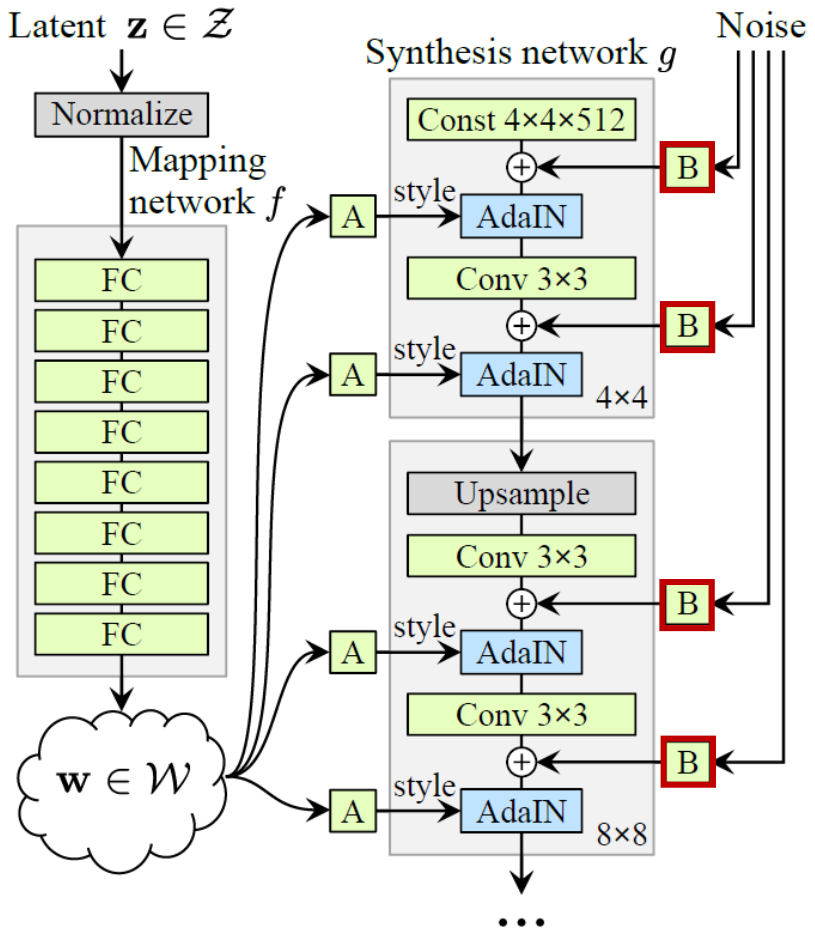
细粒度/高频特征：

- 纹理
- 眼睛高光
- 发丝细节
- 妆容边界等

低层 → 高层

西北工業大學

Style-based generator



Our code

```python
class InjectNoise(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(1, channels, 1, 1))
    def forward(self, x):
        noise = torch.randn((x.shape[0], 1, x.shape[2], x.shape[3]), device=x.device)
        return x + self.weight * noise
```

	Method	CelebA-HQ	FFHQ
A	Baseline Progressive GAN [30]	7.79	8.04
B	+ Tuning (incl. bilinear up/down)	6.11	5.25
C	+ Add mapping and styles	5.34	4.85
D	+ Remove traditional input	5.07	4.88
E	+ Add noise inputs	**5.06**	4.42
F	+ Mixing regularization	5.17	**4.40**

Random noise 让图片质量 up

西北工业大学

Problem

伪影（Artifacts）

每层的 AdaIN 会将上一层的特征图强行调整到

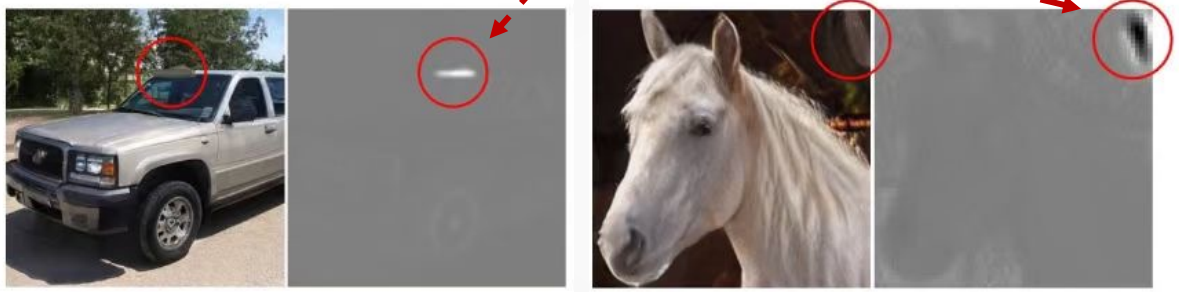
$\mu = 0$，$\sigma = 1$ 后重新计算

⇩

网络自己学到的统计结构被破坏

⇩

生成器试图通过制造“水滴状伪影”强行传递信息



Solution

权重解调（Weight Demodulation）

不再在特征图上归一化

直接在卷积核权重上做调制与解调

$$W'_{ijk} = s_i \cdot W_{ijk}$$

$$\hat{W}_{ijk} = \frac{W'_{ijk}}{\sqrt{\sum_i (W'_{ijk})^2 + \epsilon}}$$

选取需要关注的通道

⇩

加大该通道的权重
减弱其余通道的权重

⇩

通道间比例均衡而不固定

对每个通道做标准差归一

⇩

保证各通道的输出
在尺度上被约束

⇩

避免了极端亮点的产生

每层的输出保持自然的动态范围，伪影显著减少

西北工業大學

Problem

相位错位

渐进式生长训练中

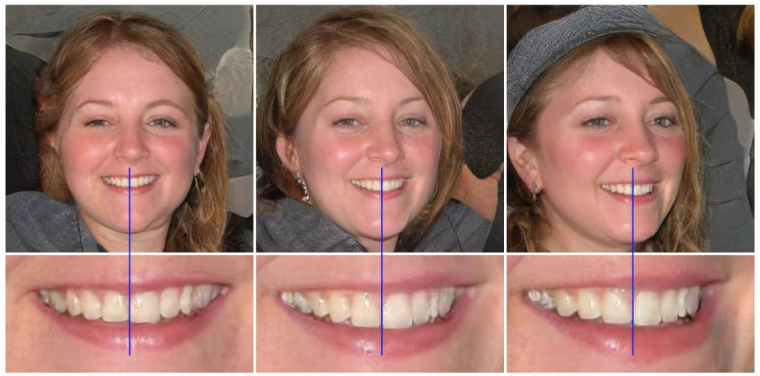
模型由低分辨率开始训练，逐步增加分辨率

⇩

新层学到的细节无法与旧层学到的全局结构保持对齐

⇩

脸在移动或缩放的时候，五官错位



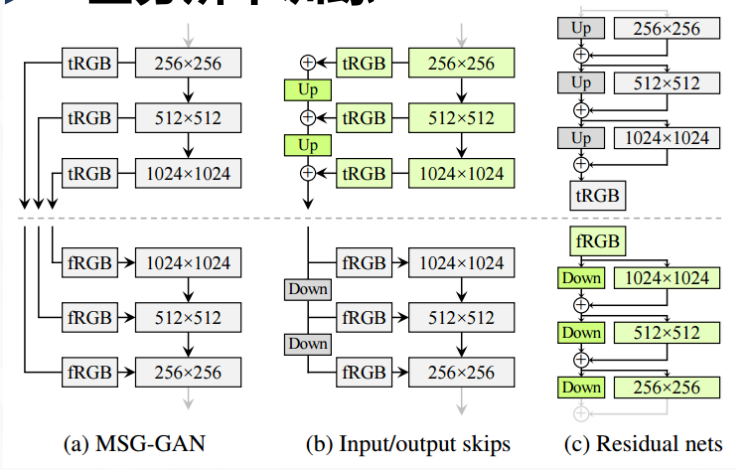
很明显

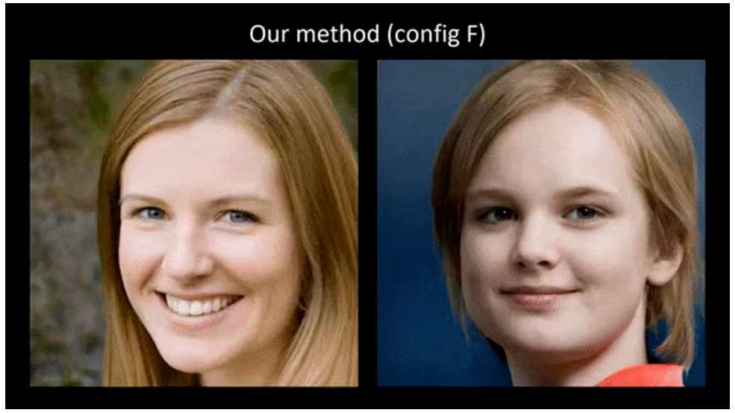
脸在转动

牙齿却保持不动

Solution

全分辨率训练





借鉴 MSG-GAN

⇩

对生成器和判别器

尝试原始结构

跳层连接和残差结构

⇩

生成器用跳层连接

判别器用残差结构

⇩

成功解决

人脸生成图像中

五官不同步的问题

西北工業大學

Problem

▶ 纹理粘附

StyleGAN2“粗糙”的信号处理过程

导致高频信号被错误“折叠”回低频区域，发生混叠

⇩

特征图中出现与“像素位置”相关的假信号

⇩

网络在学习细节纹理时，偷偷利用假信号

认为“某个像素坐标”处应有某种纹理

⇩

形成屏幕坐标依赖

纹理细节不再与物体表面绑定

⇩

脸部旋转或移动时，网络仍在固定位置输出细节

西北工業大學

Solution1

设计滤波器

信号临时上采样，提高分辨率

⇩

应用非线性，如LeakyReLU

⇩

进行低通滤波，去除伪高频信号

⇩

下采样至原分辨率

	Configuration	FID↓	EQ-T↑	EQ-R↑
A	StyleGAN2	5.14	–	–
B	+ Fourier features	4.79	16.23	10.81
C	+ No noise inputs	4.54	15.81	10.84
D	+ Simplified generator	5.21	19.47	10.41
E	+ Boundaries & upsampling	6.02	24.62	10.97
F	+ Filtered nonlinearities	6.35	30.60	10.81
G	+ Non-critical sampling	4.78	43.90	10.84
H	+ Transformed Fourier features	4.64	45.20	10.61
T	+ Flexible layers (StyleGAN3-T)	4.62	63.01	13.12
R	+ Rotation equiv. (StyleGAN3-R)	**4.50**	**66.65**	**40.48**

FID：生成图与真实图差异

EQ-T：平移等变性

EQ-R：旋转等变性

Solution2

连续信号解释

网络中的特征图不再看作“像素格子”

而是连续信号在离散网格中的采样

⇩

卷积、非线性、上下采样等操作

都在连续域里定义

⇩

最后映射回离散网格

⇩

保证网络对平移、旋转都是等变的

细节会跟着物体一起动

FID指标虽略降，但生成图结构性明显更强

西北工業大學

Environment

硬件环境

CPU 24Cores

NVIDIA GeForce 4060 Laptop GPU

软件依赖

- 操作系统 Windows 11
- CUDA-11.8.0
- Python-3.8
  - matplotlib=3.4.2
  - imageio=2.9.0
  - imgui==1.3.0
  - glfw==2.2.0
  - pyopengl==3.1.5
  - imageio-ffmpeg==0.4.3
  - pyspng
  - numpy>=1.20
  - click>=8.0
  - pillow=8.3.1
  - scipy=1.7.1
  - requests=2.26.0
  - tqdm=4.62.2
  - ninja=1.10.2

Parameters

- seed：from 000 to 008

随机数生成器的种子
控制模型在潜空间 $\mathcal{Z}$ 中采样的随机向量 z

- trunc：from 0.1 to 1.0 , stride = 0.1

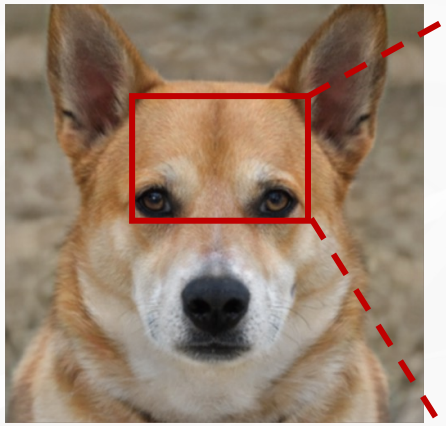
截断系数
控制潜空间 $\mathcal{W}$ 中图像的多样性和保真度平衡

- noise_mode：

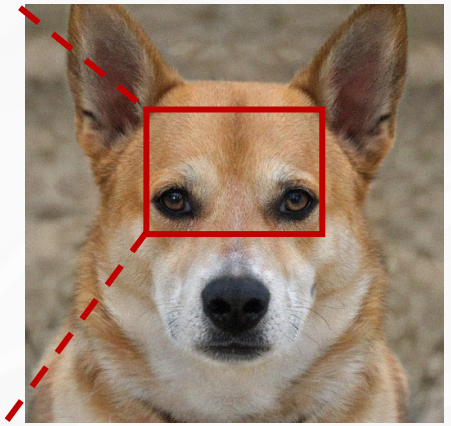
'const' - 每层像素噪声用固定常数
'random' - 每层像素噪声每次都重新随机采样
'none' - 关闭像素噪声注入

04 Our Experiments —— StyleGAN3

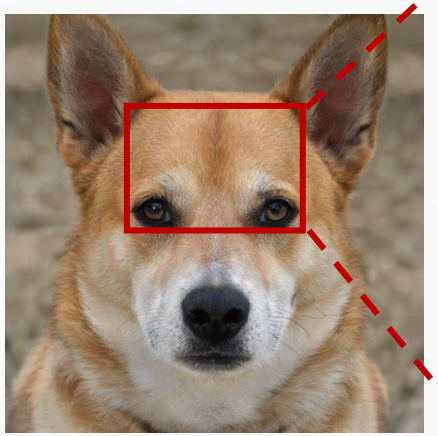
西北工業大學



None



Random

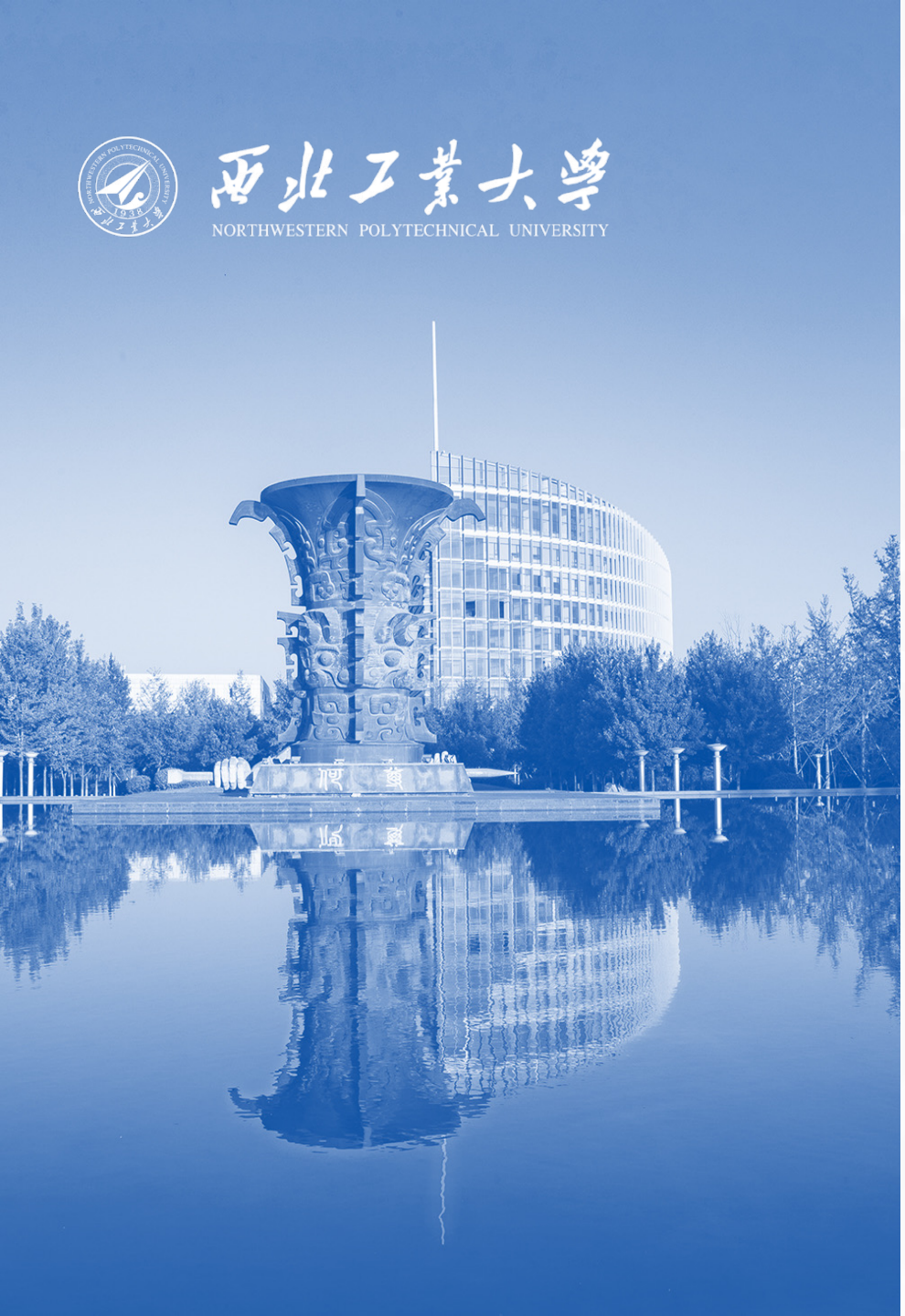


Const

不难发现，在毛发细节方面：

添加固定/随机噪声生成的图片信息更丰富！

添加随机噪声生成的图片更逼真！



请老师批评指正！

汇报人：xxxxx

汇报时间：2025.10.28